

tf.TensorSpec Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/TensorSpec

Describes the type of a tf.Tensor.

Function Signatures

```
tf.TensorSpec( shape, dtype=tf.dtypes.float32, name=None )
```

Code Examples

```
tf.TensorSpec(  
    shape,  
    dtype=tf.dtypes.float32,  
    name=None  
)
```

```
tf.TensorSpec(  
    shape,  
    dtype=tf.dtypes.float32,  
    name=None  
)
```

```
tf.dtypes.float32
```

```
t = tf.constant([[1,2,3],[4,5,6]])  
tf.TensorSpec.from_tensor(t)  
TensorSpec(shape=(2, 3), dtype=tf.int32, name=None)
```

```
t = tf.constant([[1,2,3],[4,5,6]])
```

```
tf.TensorSpec.from_tensor(t)
```

```
TensorSpec(shape=(2, 3), dtype=tf.int32, name=None)
```

```
@tf.function(input_signature=[tf.TensorSpec([1, None])])  
def constrained_foo(t):  
    print("tracing...")  
    return t
```

Documentation

Describes the type of a `tf.Tensor`.

Inherits From: `TypeSpec`, `TraceType`

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.TensorSpec`

Used in the notebooks

- Extension types
 - Import a JAX model using JAX2TF
 - Using the SavedModel format
 - Better performance with `tf.function`
 - Migrate the SavedModel workflow
 - DeepDream
 - Simple audio recognition: Recognizing keywords
 - Load video data
 - Transfer learning for video classification with MoViNet
 - Video classification with a 3D convolutional neural network

Contains metadata for describing the nature of `tf.Tensor` objects accepted or returned by some TensorFlow APIs.

For example, it can be used to constrain the type of inputs accepted by a `tf.function`:

Now the `tf.function` is able to assume that `t` is always of the type `tf.TensorSpec([1, None])` which will avoid retracing as well as enforce the type restriction on inputs.

As a result, the following call with tensor of type `tf.TensorSpec([1, 2])` triggers a trace and succeeds:

The following subsequent call with tensor of type `tf.TensorSpec([1, 4])` does not trigger a trace and succeeds:

But the following call with tensor of type `tf.TensorSpec([2, 2])` fails:

Raises

`## Attributes`

Methods

`## experimental_as_proto`

[View source](#)

Returns a proto representation of the `TensorSpec` instance.

experimental_from_proto

[View source](#)

Returns a TensorSpec instance based on the serialized proto.

experimental_type_proto

[View source](#)

Returns the type of proto associated with TensorSpec serialization.

from_spec

[View source](#)

Returns a TensorSpec with the same shape and dtype as spec.

from_tensor

[View source](#)

Returns a TensorSpec that describes tensor.

is_compatible_with

[View source](#)

Returns True if spec_or_tensor is compatible with this TensorSpec.

Two tensors are considered compatible if they have the same dtype and their shapes are compatible (see `tf.TensorShape.is_compatible_with`).

is_subtype_of

[View source](#)

Returns True if self is a subtype of other.

Implements the `tf.types.experimental.func.TraceType` interface.

If not overridden by a subclass, the default behavior is to assume the `TypeSpec` is covariant upon attributes that implement `TraceType` and invariant upon rest of the attributes as well as the structure and type of the `TypeSpec`.

`most_specific_common_supertype`

[View source](#)

Returns the most specific supertype `TypeSpec` of self and others.

Implements the `tf.types.experimental.func.TraceType` interface.

If not overridden by a subclass, the default behavior is to assume the `TypeSpec` is covariant upon attributes that implement `TraceType` and invariant upon rest of the attributes as well as the structure and type of the `TypeSpec`.

`most_specific_compatible_type`

[View source](#)

Returns the most specific `TypeSpec` compatible with self and other. (deprecated)

Deprecated. Please use `most_specific_common_supertype` instead.

Do not override this function.

`__eq__`

[View source](#)

Return self==value.

`__ne__`

[View source](#)

Return self!=value.